Guarded Research-Agent Loops with Swarm Collapse and Web-Scale Acquisition

Working manuscript · July 2026

Abstract

Research agents combine language-model inference with tool execution, web retrieval, local state, and iterative refinement. Their failure modes differ from ordinary batch classifiers: a single unsafe tool call can mutate a workspace, a credential leak can enter persistent state, and a multi-agent run can multiply model cost by the number of agents and deliberation rounds. This paper describes a guarded research-agent system built from four components. `syndesi` provides the model/tool loop, identity checks, permission gates, guardrail hooks, MCP federation, session accounting, and threat normalization. `webfold` provides the acquisition layer: live URL fetch, same-host crawling, headless browser/CDP operation, DOM-tree construction, HTTP proxying, static HTML extraction, WARC and `.seed` archive processing, and document conversion through adjacent fold-family tools such as `pdffold`. The memory substrate represents session and research state as append, branch, fold, and graph operations over classified records. Swarm collapse provides a conditional fixed-point reduction for multi-agent dynamics that satisfy a contraction criterion.

The paper gives source-level system properties for `syndesi`, the acquisition and memory interface contracts, mathematical conditions for swarm collapse, and measured benchmark results for one `webfold` acquisition mode: Common Crawl WARC processing. On a 15.9 GB uncompressed WARC subset containing 84,749 HTTP response records, the fastest measured local path processes a 4.95 GB `.seed` artifact in 6.6 s wall time, with 3.17 s spent on decompression, extraction, and Markdown rendering. This corresponds to 26,675 pages/s and 3,585 MB/s of HTML extraction throughput. The raw WARC 8-thread path processes all 84,749 pages in 16.9 s at 14,238 pages/s. Single-threaded `.seed` streaming obtains 2,943 pages/s at 35 MB RSS. The S3 streaming path is network-bound at approximately 67 MB/s from Tasmania to Wasabi Sydney, reaching 1,169 pages/s with eight parallel TLS connections. Compared with Daft 0.7.20 on the same corpus, `webfold` is 39.9x faster on the full parse/filter/extract/render pipeline when Daft uses a Python UDF for HTML extraction.

The system separates four measurable problems. Acquisition is evaluated by surface-specific variables: browser/CDP capability, DOM observability, document coverage, WARC throughput, transport bandwidth, memory, and page coverage. Agent action is evaluated as a gated state-transition problem. Memory is evaluated as append, branch, fold, and graph operations over classified records. Multi-agent fanout is evaluated as a dynamical systems problem: when the iterated update $S_{t+1} = AS_t + B$ satisfies $\rho(A) < 1$, the converged state is replaced by the fixed point $S^* = (I - A)^{-1}B$; when the condition fails, the cluster is simulated explicitly.

---

1. Introduction

A research agent is a closed-loop state machine. It receives an input, queries a model, parses an action, executes a tool, appends the result to state, and queries the model again. The loop is useful because it can inspect files, run experiments, search corpora, and repair intermediate errors. The same loop is risky because tool calls have side effects and because the model can observe, persist, or emit sensitive data.

This paper starts from that minimal loop. Let x_t denote the session state at iteration t, including the prompt, prior messages, tool results, retrieved documents, and accounting metadata. A language model proposes an action $a_t = \pi_\theta(x_t)$. The runtime checks the action, executes it if allowed, and appends an observation o_t. The next state is

$$x_{t+1} = T(x_t, a_t, o_t).$$

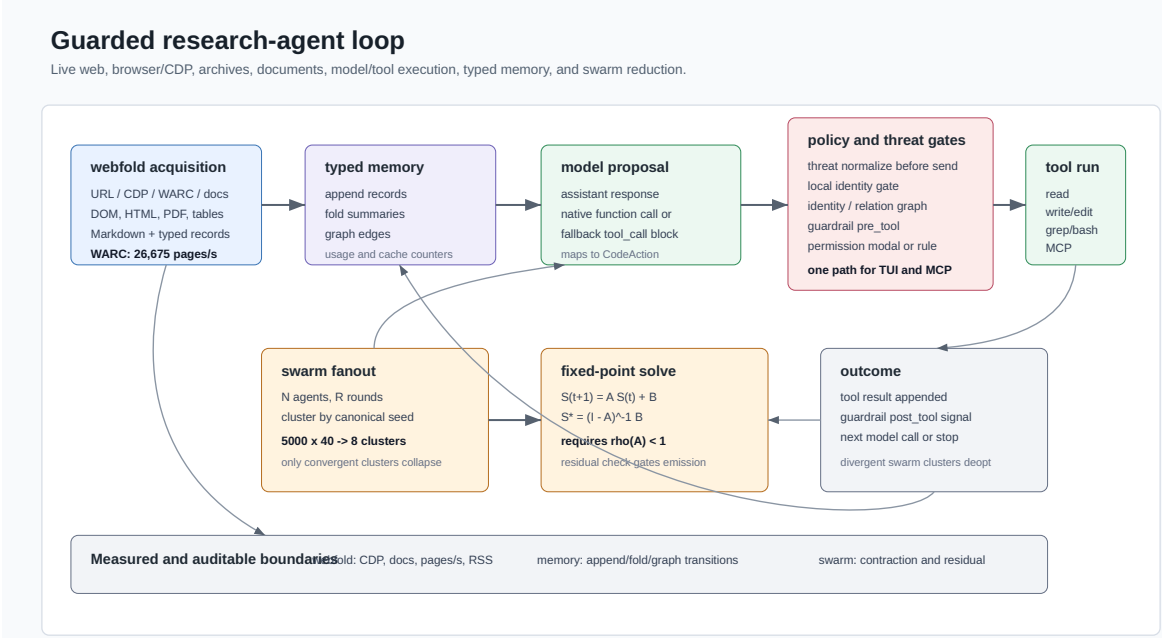


Figure 1: Research-agent loop, swarm-collapse boundary, and webfold acquisition path.

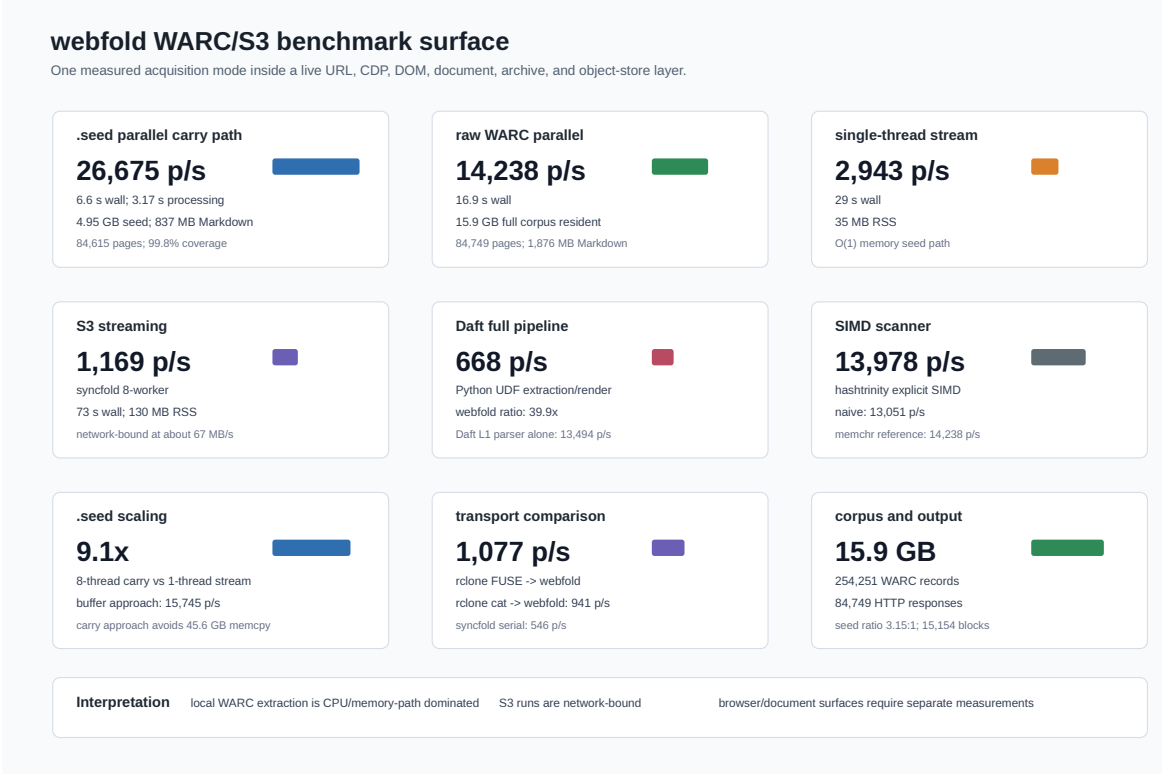


Figure 2: webfold WARC/S3 benchmark surface.

The engineering problem therefore includes more than model quality. The runtime must define which transitions are legal, which observations are persisted, how external data enters the loop, and how repeated agents are reduced when a research plan fans out across many similar tasks.

Web research adds an acquisition problem. Agents need live pages, web archives, browser state, DOM trees, network traces, document collections, tables, and extracted text. Browser/CDP crawling captures dynamic pages and user-agent dependent behavior. Static archive processing is faster for fixed corpora, but it must report coverage and format-boundary losses. Document conversion must also be treated as acquisition: a PDF, CSV, spreadsheet, or office document is an external state surface whose typed content must be normalized before it enters the agent context.

Multi-agent research adds a cost problem. A naive swarm of N agents running R rounds costs $O(NR)$ model calls or policy updates. If the per-cluster state update is affine and contractive, the converged state can be computed directly from a fixed point. If the contraction condition is not satisfied, the reduction is invalid and the system must fall back to explicit iteration.

This manuscript evaluates the complete loop through four components:

1. `webfold` acquires live web pages, browser/CDP observations, WARC/`.seed` archives, S3-backed corpora, and document-derived text/table records.
2. `syndesi` runs a guarded model/tool loop over local and federated tools.
3. The memory substrate stores session state as append-only records, fold summaries, classified branches, and graph edges.
4. Swarm collapse reduces convergent multi-agent fanout to clustered fixed points.

The contribution is a systems formulation for research agents in which acquisition, tool execution, memory, security, and multi-agent fanout have explicit variables, metrics, and refusal conditions.

2. Evaluation Design

The paper evaluates eight claims. They are not combined into a single score, because they have different units of evidence. Some claims are source-level contracts, such as whether mutation-capable tools share one authorization path. Some are throughput measurements, such as WARC pages per second. Some are mathematical refusal conditions, such as whether a multi-agent update is contractive. Treating those as one benchmark would hide the variable being tested.

Claim C1 is that model-proposed actions enter one gated transition path before they can mutate local state or call an external tool. The test is source inspection over the action parser, native tool dispatcher, fallback tool-call handler, TUI path, MCP gateway, identity checks, relation-graph checks, guardrail hooks, and permission rules. The claim fails if any mutation-capable path bypasses that gate.

Claim C2 is that threat normalization occurs before model exposure and before durable record writes. The test is ordering: raw input must pass through the threat-law scanner and span normalizer before it is appended to session state, sent to the model, or written to a persistent record. The claim fails if a raw secret or PII span reaches model context or durable state before normalization.

Claim C3 is that acquisition is a typed-observation interface rather than a single file parser. The test is whether live URL fetch, same-host crawl, headless/browser state, CDP-compatible automation, DOM-tree construction, proxying, static HTML, WARC/`.seed`, S3-backed archive streams, PDF conversion, and table/document inputs can all emit records with source identity, content type, normalized spans or rows, provenance links, and downstream Markdown/chunk/table representation. The claim fails if one acquisition surface requires a different agent-loop contract.

Claim C4 is that the native archive extractor processes Common Crawl response pages at tens of thousands of pages per second on commodity hardware. The test uses a 15.9 GB raw WARC corpus and a 4.95 GB compressed `.seed` artifact. The measured variables are pages/s, wall time, processing time, HTML MB/s, peak RSS, page coverage, and Markdown output size.

Claim C5 is that remote S3-backed archive processing is transport-bound when object-store bandwidth is below local extraction throughput. The test streams the same `.seed` artifact over a Wasabi S3 path from Tasmania to Sydney and compares `syncfold s3://`, rclone FUSE, rclone pipe, and the local extraction rate. The claim fails if remote extraction CPU, rather than object-arrival rate, is the limiting variable.

Claim C6 is that a dataframe WARC parser plus Python UDF extraction is limited by the UDF boundary at full-pipeline level. The test separates Daft native WARC parsing from Daft Python `.apply()` extraction and compares the full parse/filter/extract/render task against native `webfold`.

Claim C7 is that conversation and research memory can be represented with append, read, branch, fold, and graph primitives rather than a flat transcript buffer. The test is source and interface inspection: records must carry identity, parentage, classification, payload, provenance, and graph relationships sufficient for branches, summaries, counters, permissions, source chunks, and task dependencies.

Claim C8 is that swarm collapse is valid only for clustered agent dynamics that satisfy a contraction condition. The test is mathematical: for an affine cluster update $S_{t+1} = AS_t + B$, collapse is allowed only when $\rho(A) < 1$ and the residual check passes. Otherwise the cluster must deopt to explicit iteration.

The materials follow those claims. `syndesi` source and design notes support C1, C2, and C7. `webfold` source modules, the `pdffold` converter, and the document/table adapter contract support C3. The Common Crawl WARC, compressed `.seed`, Wasabi object-store path, rclone runs, and Daft runs support C4-C6. The swarm-collapse recurrence and residual check support C8. Codex, Claude Code, OpenCode, and OpenClaw documentation is used only to characterize the adjacent-agent baseline for C1/C7: those systems are coding or personal-agent surfaces, while this paper requires a research harness in which acquisition, memory, policy, threat normalization, cost accounting, MCP federation, and external agent multiplexing are all visible as state transitions.

The WARC benchmark is intentionally narrow. It does not measure browser visual fidelity, JavaScript execution equivalence, login flows, screenshot capture, PDF layout recovery, spreadsheet formula semantics, or adversarial document parsing. Those are acquisition surfaces under C3, but the measured throughput claims in this paper are C4-C6: static archive extraction and remote archive streaming.

3. System Model

3.1 Agent State Transition

An agent runtime can be represented as a controlled transition system:

$$(x_t, u_t) \to (x_{t+1}, y_t),$$

where x_t is session state, u_t is the user or tool observation entering the loop, and y_t is the emitted assistant message or tool call. A transition is valid only when the proposed action passes the runtime policy.

The relevant state fields are:

Field	Description
message log	user, assistant, and tool messages exposed to the model
local workspace	files and command outputs reachable by tool execution
acquired corpus	URLs, DOM nodes, screenshots or page observations, WARC records, Markdown blocks, tables, document spans
identity state	local actor identity and remote authorization metadata
policy state	permission rules, guardrail decisions, denied actions

Field	Description
accounting state	model provider, token counts, cost estimates, cache state
memory state	append-only records, branch links, fold summaries, graph edges, typed extraction records

The central safety requirement is ordering. Threat scanning must occur before a span reaches the model or persistent state. Authorization must occur before a mutating tool executes. Tool results must be recorded after execution and before the next model call so that the model cannot operate on an untracked side effect.

3.2 Acquisition Surfaces

The acquisition path converts external web and document material into deterministic agent context:

```
inquiry
  -> typed acquisition schema
  -> live URL / browser / WARC / document adapter
  -> DOM tree, HTTP body, rendered observation, table, or text spans
  -> Markdown/table/record rendering
  -> chunking and session insertion
```

`webfold` exposes multiple acquisition surfaces. The native fetch path maps a URL to an HTTP response with user-agent and proxy controls. The crawl path fetches a URL frontier and folds repeated site chrome into a template table. The headless path builds a DOM tree, supports CDP-compatible navigation, runtime evaluation, DOM reads, input events, and browser observations. The archive path reads WARC and `.seed` artifacts from local disk or object storage. The document path delegates PDF conversion through `pdffold` and normalizes document text into the same downstream chunking and record interface. CSV, spreadsheet, and office-document adapters fit the same contract: decode a file surface into typed rows, cells, or spans before the agent sees it.

The benchmarked external objects are an uncompressed Common Crawl WARC file, a compressed `.seed` artifact, and an S3-compatible object containing the same seed. A WARC file is a sequence of records; only HTTP response records contribute page bodies in the reported page/s measurements. The `.seed` representation stores compressed blocks, enabling streaming decompression or parallel block processing.

3.3 Memory and Database Model

The agent memory model is an applicative record system rather than a flat transcript buffer. The primitive operations are:

Primitive	Operation	Agent-level use
append	add a classified record under an optional parent	user message, assistant message, tool call, extraction result
read	fetch one record by identifier subject to classification	replay, audit, citation
read branch	walk parent links from a leaf to a root	restore a conversation or task trajectory
fold	summarize or accumulate records under a boundary	compaction, cost counters, cache reads, schema aggregation
graph edge	relate records, tools, permissions, sources, and tasks	authorization, provenance, work graph, revocation
query	select nodes/edges by kind, time, source, or classification	retrieval, filtering, audit, swarm clustering

This model separates immutable audit events from conversation continuation. Summary records are not deleted history; they are continuation seeds with explicit boundaries. Tool results are not side-channel output; they are records whose edges connect the action, permission decision, source artifact, and subsequent model call.

Compared with CxDB-style context storage, the relevant distinction is the query algebra. CxDB's conversation API is organized around contexts and turns: append a turn, read a turn, and walk a branch through parent links. That shape is useful for conversation history. The fold/graph model uses the same append and branch operations but additionally treats summaries, counters, permissions, source chunks, typed extraction schemas, and work dependencies as graph nodes with edge-reading queries. It therefore supports conversation replay and agent-governance queries through the same substrate.

3.4 Multi-Agent Fanout

For repeated agent runs, the naive execution cost is

$$C_{\text{naive}} = N \cdot R \cdot C_{\text{step}},$$

where N is the number of agents, R is the number of rounds, and C_{step} is the cost of one model or policy update. Fanout is reduced only when agents can be clustered into equivalent update dynamics. If cluster k has state S_k and update

$$S_{k,t+1} = A_k S_{k,t} + B_k,$$

then repeated iteration has a unique fixed point when $\rho(A_k) < 1$. The fixed point is

$$S_k^* = (I - A_k)^{-1} B_k.$$

The runtime must check the residual

$$\|S_k^* - (A_k S_k^* + B_k)\|$$

and deopt any cluster that fails the condition or residual threshold.

4. Component Design

4.1 Syndesi: Guarded Tool Loop

`syndesi` is a Rust package whose binary is `gateway`. It provides a textfold-based TUI, an MCP server surface, provider configuration, native tool execution, usage accounting, threat scanning, identity checks, relation-graph authorization checks, and guardrail policy hooks. The runtime is designed as a research harness rather than only a coding assistant: it coordinates local files, shell commands, MCP tools, provider calls, acquisition tools, memory records, cost counters, threat findings, and external agent sessions through one loop.

The source-level tool loop has two model-facing input forms. A model can emit a native tool call, or it can emit a fallback text block:

```
<tool_call name="grep">
pattern=SeedStream
include=rs
</tool_call>
```

Both forms map into the same `CodeAction` variants:

Action	Effect
`Read`	read one or more files
`Write`	write a file
`Edit`	replace text in a file
`Glob`	enumerate paths
`Grep`	search file contents
`Bash`	run a shell command
`Think`	append private reasoning text as a tool result

The execution path is:

```
model output
  -> native tool call or parse_tool_call
  -> CodeAction
  -> local identity gate
  -> identity / relation-graph / guardrail pre_tool check
  -> permission rule or modal decision
  -> ToolRunner::run_with_permissions
  -> guardrail post_tool signal
  -> tool result appended to session
  -> next model call
```

This path is a state-machine boundary. The model can propose an action, but the runtime decides whether the transition is permitted. Interactive TUI calls and `gateway mcp-serve` calls share this boundary. MCP federation extends the tool catalog, but the federated call still enters the same permission and policy stack before execution.

4.2 Why This Harness

Current agent harnesses occupy related but different design points. Codex is available through CLI, IDE, and cloud surfaces and exposes approval and sandboxing controls in its product documentation. Claude Code is described by Anthropic as an agentic coding tool that reads codebases, edits files, runs commands, integrates with development tools, and connects to external tools via MCP. OpenCode is an open-source AI coding agent exposed as a terminal interface, desktop app, and IDE extension. OpenClaw is a self-hosted personal assistant that runs on user devices, answers through existing communication channels, and emphasizes persistent local assistant behavior. These systems are useful baselines, but their public surfaces primarily optimize for interactive coding, personal-assistant orchestration, or productized hosted execution.

`syndesi` is selected for this paper because the research loop requires four properties at the same time:

Requirement	Why it matters for this paper	Syndesi mechanism
one tool path	TUI, native model calls, fallback text calls, and MCP tools must be comparable	all actions map to `CodeAction` and `ToolRunner::run_with_permissions`
provider neutrality	benchmark and research loops should not depend on one model vendor	provider configuration is local runtime state
acquisition-native tools	web, WARC, PDF, chunks, security scans, and local files are first-class inputs	fold-family tools are invoked as local tool substrates
memory as records and graph	research state needs provenance, branches, summaries, permissions, and source edges	append/read/fold/graph primitives rather than an opaque transcript

The comparison is therefore architectural. Codex and Claude Code supply strong coding-agent surfaces. OpenCode supplies an open terminal coding-agent surface. OpenClaw supplies a persistent personal-agent orchestration model. The system studied here needs the gateway itself to be the experimental object: the same runtime must acquire web/document corpora, normalize sensitive spans, execute tools under a policy gate, persist typed records, and collapse repeated agent fanout. That requirement makes `syndesi` the controllable harness for the reported method.

4.3 Threat Normalization

Before a user prompt reaches the model, session history, persistent event records, or input history, `syndesi` applies a threat scan. A `Law` is a domain-specific scanner whose violations are spans. `scan_all` collects findings and `normalize` redacts overlapping spans using a longer-span-wins rule.

The default law set includes secrets and PII-like entities. The secret law covers prefix-anchored tokens such as OpenAI, Anthropic, GitHub, AWS, Slack, Google, and Stripe key shapes; key/value assignment forms such as `api_key=...`; and private-key blocks. The PII law follows the `pii-svc` entity vocabulary and checksum-style validators.

The input path is therefore:

```
raw input -> scan laws -> normalize spans -> append redacted input -> send
```

Clean input is preserved. Inputs with findings are replaced by explicit redaction tokens before persistence or model exposure. This ordering matters: redaction after model submission would protect only later logs, and redaction after record persistence would leave durable sensitive state behind.

4.4 Webfold: Browser, Archive, and Document Acquisition

`webfold` spans web acquisition, browser state, archive processing, and document conversion. Its core library walks HTML bytes once, slices tag and attribute regions, emits Markdown blocks, and computes normalized block signatures. A crawl template is a frequency table over block signatures; repeated chrome is removed by table lookup. Around that core are live fetch, crawl, proxy, headless browser, CDP, DOM, archive, and document surfaces.

Surface	Operation	Agent-visible output
live URL fetch	`FetchBackend::fetch` with user-agent, redirect, timeout, and proxy options	HTTP body, links, Markdown
same-host crawl	fetch frontier, parse each page, fold repeated template blocks	content-only Markdown per page
headless/browser	navigate, build DOM tree, evaluate runtime expressions, query DOM, dispatch input	DOM nodes, text, events, screenshot/page observations where the capture backend is enabled
CDP server	Playwright/Puppeteer-compatible WebSocket subset for Page, DOM, Runtime, Network, Input, Target/Session paths	automation-compatible page state
WARC and `.seed`	local raw WARC, compressed seed blocks, S3-backed stream	response pages, Markdown, throughput counters
PDF/documents	`pdffold` bridge for PDF; ODT/CSV/XLSX-style document and table adapters map files to text spans or typed rows	Markdown, chunks, table records

The WARC pipeline reads response records, extracts HTTP bodies, runs `to_blocks`, and renders Markdown. The live-web pipeline can instead fetch a URL directly, crawl a site, or expose a CDP endpoint to a browser automation client. The document pipeline converts non-HTML inputs into the same downstream record interface.

The common interface is a typed observation stream rather than a file-format boundary. Each acquisition path emits records with source identity, content type, normalized spans or rows, provenance links, and optional browser state. This lets a research run combine live pages, archived pages, rendered DOM state, screenshots or page observations, PDFs, office documents, spreadsheets, and CSV tables without changing the agent-loop contract.

The benchmarked pipeline has three relevant input modes:

Mode	Representation	Memory behavior
raw `.warc` parallel	uncompressed WARC bytes	reads the full corpus for byte-range partitioning
`.seed` streaming	compressed seed blocks	O(1) memory in the single-thread stream
`.seed` parallel	compressed seed blocks in memory	block-parallel decompression and carry-based WARC alignment

The `.seed` path uses `hashtrinity::codec::lz::compress_turbo` with 1 MB blocks. The optimized parallel path processes decompressed blocks immediately and carries only the partial WARC record tail across block boundaries, avoiding per-thread construction of a full decompressed buffer.

4.5 Swarm Collapse

Swarm collapse applies when many agents share equivalent update dynamics after canonicalization. The protocol is:

1. Cluster agents by canonical seed, persona, and action parameters.
2. Estimate or read the affine update for each cluster.
3. Check the contraction condition and residual against the iterative baseline.
4. Emit the fixed point for convergent clusters.
5. Deopt divergent clusters to explicit simulation or model calls.

The local swarm-collapse note reports 5,000 agents and 40 rounds reduced to 8 clusters and one fixed-point pass, with approximately 10^{-7} residual in the contractive case. The step reduction is approximately 25,000x relative to 5,000 agents times 40 rounds. This reduction is conditional; the fixed-point solve replaces convergent iteration only.

5. Experimental Method

5.1 Webfold Benchmark Environment

The webfold benchmark was run on `root@10.0.63.43` in Tasmania, Australia, on 2026-07-15.

Component	Specification
CPU	AMD Ryzen 7 5700X, 8 cores / 8 threads, BIOS-limited QEMU at 2.0 GHz
L1 cache	512 KiB total, 8 x 64 KiB per core
L2 cache	4 MiB total, 8 x 512 KiB per core
L3 cache	128 MiB shared
RAM	71 GiB DDR4

Component	Specification
GPU	NVIDIA GeForce RTX 3060, 8 GiB VRAM, driver 580.159.03, CUDA 12.0
Disk	600 GB QEMU virtual disk backed by NVMe SSD
Network	approximately 27 ms RTT to Wasabi S3 Sydney
OS	Ubuntu 24.04.4 LTS, Linux 6.8.0-124-generic
Rust	rustc 1.97.0, cargo 1.97.0
rclone	1.74.4
Daft	0.7.20, Python 3.12, pyarrow 24.0.0

The VM used QEMU/KVM with no CPU pinning. OS page cache was warm for local disk runs. The guest did not expose CPU governor and frequency scaling controls.

5.2 Dataset

Property	Value
Source	Common Crawl subset of scraped web pages
Raw WARC size	15.9 GB
Total WARC records	254,251
HTTP response records	84,749
Average response HTML size	approximately 183 KB
`.seed` size	4.95 GB
Compression ratio	3.15:1
Seed blocks	15,154
Markdown output, raw full extraction	1,876 MB
S3 location	Wasabi `ptyktos-webscraping-dev/cc-warc/full-corpus.warc.seed`

The corpus is alphabetically sorted by URL and includes English, Chinese, Japanese, Russian, Polish, German, Arabic, and other languages. Page types include blogs, e-commerce, government pages, academic pages, news, and social media.

The benchmark uses the same Common Crawl-derived object in two operational modes. The local mode downloads or stages the WARC/seed before processing; this isolates CPU, memory, decompression, record alignment, and renderer cost. The remote mode treats the object store as attached storage and streams the seed over an S3-compatible API; this measures the acquisition path when the corpus is too large or too operationally expensive to pre-stage. The Wasabi run used a Tasmania-to-Sydney path with approximately 27 ms round-trip time. In production collection, the same S3-compatible object store can be the sink for generated WARC/seed artifacts and the source for downstream parsing jobs.

For new collection rather than fixed-corpus evaluation, the same acquisition layer can start from an inquiry-derived typed schema. The schema defines the records to extract, such as URL, title, timestamp, page text, DOM selectors, tables, author fields, citation metadata, prices, or vulnerability findings. Live crawling, CDP automation, WARC generation, and document conversion then populate rows or graph nodes under that schema. Public object-store corpora such as research metadata dumps can be processed by the same predownload or direct-stream pattern, avoiding rate-limited per-record API calls when a bulk object is available.

5.3 Measurements

The webfold measurements report:

Metric	Definition
Pages/s	HTTP response records processed per second
Wall time	total command duration where reported
Processing time	decompression, extraction, and Markdown rendering time
HTML extraction rate	input HTML bytes processed per second
RSS	peak resident set size
Page coverage	processed response pages divided by response records
Markdown output	rendered Markdown bytes

The Daft comparison separates native WARC parsing from full extraction because Daft's native `read_warc` path is much faster than the Python UDF used for HTML body extraction and tag stripping.

5.4 Fairness and Threats to Validity

The benchmark measures systems throughput for the WARC/S3 surface. It reports the rate of a static extractor on WARC response bodies and the cost of moving a seed object from S3 into the extractor. Page-quality measurements, visual browser equivalence, JavaScript execution, login flows, screenshot capture, and adversarial HTML are handled by the live browser/CDP surface and require separate benchmarks. They are not implied by the WARC numbers.

The Daft UDF comparison should be read as a full-pipeline comparison against the tested implementation. Daft's native WARC parser is fast. A native HTML extractor inside Daft would change the result. The reported ratio isolates the effect of falling from a native parser into a per-row Python extraction path.

The `.seed` carry path is the fastest local path but loses 134 pages at seed block boundaries in the reported run. The serial streaming path does not have that loss. Exact-coverage runs should use an alignment mode that preserves records across seed boundaries or should report the observed loss.

6. Results

Results are reported by claim. Source-level claims are resolved first because they define which transitions and observations are legal. Throughput claims then measure one acquisition surface, the WARC/S3 archive path.

6.1 C1-C2: Gated Action and Pre-Exposure Normalization

The inspected agent loop does not treat model output as an executable command. The runtime first parses the proposal into a tool action or assistant message. Mutation-capable actions then pass through the same policy boundary used by the TUI and MCP-facing tool surfaces:

```
model proposal
  -> parsed action
  -> CodeAction or tool request
  -> identity / relation-graph / guardrail pre_tool check
  -> permission rule or approval path
  -> local tool or MCP execution
  -> post_tool signal
  -> classified record append
```

This establishes C1 for the inspected action paths. The result is a source contract, not a performance benchmark. It means the system can be evaluated as a controlled transition system: a model proposal becomes a candidate transition, and the transition is legal only if identity, relation, guardrail, and permission

checks allow it. The claim would not hold for any plugin, connector, or external execution path that mutates state outside this boundary.

C2 is also an ordering result. Threat-law scanning and normalization occur before model-visible or durable-state exposure:

```
raw input -> scan laws -> normalize spans -> append redacted input -> model/tool exposure
```

The consequence is concrete. If an input contains a secret-like span or PII-like entity, the durable session record and the next model context receive the normalized token rather than the raw span. A sanitizer that ran only after the model call would not satisfy this claim, because the unsafe value would already have entered the model context. A sanitizer that ran only after persistence would not satisfy it either, because the raw value would already be durable.

6.2 C3 and C7: Acquisition and Memory Interfaces

The acquisition layer exposes more than WARC processing. The inspected surfaces include live URL fetch, same-host crawl, headless/browser session, CDP-compatible automation, DOM-tree construction, proxy operation, static HTML extraction, WARC and `.seed` archive processing, S3-backed archive streaming, PDF conversion through `pdffold`, and document/table adapters for ODT, CSV, XLSX, and adjacent office/table inputs.

These surfaces share one downstream contract: an acquisition adapter emits typed observations. A typed observation contains source identity, content type, normalized spans or rows, provenance links, and optionally browser state. The agent loop then consumes Markdown blocks, chunks, table records, DOM observations, HTTP bodies, or graph nodes without needing a different policy path for each source. This establishes C3 as an interface result. It does not assign WARC throughput numbers to browser rendering, screenshot capture, PDF layout recovery, or spreadsheet semantics.

The memory model establishes C7. The primitive operations are append, read, read branch, fold, and graph relation. A message, tool result, source chunk, permission decision, cost counter, extraction schema, or task dependency can be represented as a classified record plus edges. A fold summarizes or accumulates a bounded subset of records; a branch read follows parent links; a graph query follows provenance, authorization, or task edges. This is the property required by the research loop: the system can store conversation state, acquisition outputs, permission state, summaries, and work dependencies without collapsing them into a single flat transcript buffer.

This is also the selection criterion for `syndesi` as the experimental harness. Codex and Claude Code are strong coding-agent products; OpenCode is an open terminal coding-agent implementation; OpenClaw is a self-hosted personal-agent system. The experiment in this paper requires record-level control over the agent loop itself: acquisition records, model proposals, native tool calls, MCP calls, permission decisions, threat findings, cost counters, memory branches, and external agent sessions must all be inspectable as state transitions. `syndesi` is used because those variables are part of the local harness rather than opaque product behavior.

6.3 C4: Local WARC and `.seed` Processing

Path	Pages/s	Wall time	RSS	Notes
`webfold .seed` 8-thread carry path	26,675	6.6 s	4.8 GB	fastest local path; 3.17 s processing
`webfold raw WARC` 8-thread	14,238	16.9 s	15.2 GB	full corpus resident; all 84,749 pages
`webfold .seed` single-thread	2,943	29 s	35 MB	O(1) streaming

Path	Pages/s	Wall time	RSS	Notes
`webfold raw WARC` single-thread	1,996	42.5 s	15.2 GB	old baseline

The best `.seed` path reads 4.95 GB in 3.1 s, processes 84,615 pages in 3.17 s, and outputs 837 MB of Markdown. It processes 99.8% of response pages; 134 pages are lost at seed block boundaries in the carry-based WARC alignment. The raw WARC parallel path processes all 84,749 pages and outputs 1,876 MB of Markdown.

The raw WARC path is slower for two reasons. First, it reads 15.9 GB rather than 4.95 GB. Second, byte-range partitioning over the full uncompressed corpus requires the corpus to be resident, producing a 15.2 GB peak RSS. The single-thread `.seed` stream is slower than the 8-thread carry path but has a 35 MB RSS, making it the appropriate configuration when memory is the primary constraint.

Verbose URL printing reduces the `.seed` path from 26,675 pages/s to 21,776 pages/s, an 18% reduction from stderr I/O. The non-verbose result is therefore the better estimate of extraction capacity.

6.4 C5: S3 Streaming

All S3 runs stream the 4.95 GB seed from Wasabi S3 over the Tasmania-to-Sydney network path.

Configuration	Pages/s	Wall time	RSS	Notes
`syncfold s3://` 8-worker parallel	1,169	73 s	130 MB	8 TLS connections; network-bound
rclone FUSE -> `webfold`	1,077	79 s	31 MB	rclone mount, local file path
rclone cat -> `webfold`	941	90 s	31 MB	pipe from rclone
`syncfold s3://` serial	546	155 s	42 MB	single TLS connection
original broken `syncfold s3://` path	0.7	188 s+	21 MB	format decoder bypass; fixed in benchmark session

The observed network ceiling is approximately 67 MB/s, measured independently by `rclone cat | wc -c` over the same object. The eight-worker `syncfold` path, rclone FUSE path, and rclone pipe path are therefore transport-bound. A local path processes the same seed at 26,675 pages/s, whereas the best S3 path reaches 1,169 pages/s. The limiting variable is the arrival rate of compressed blocks from the object store; the local extractor has substantial processing headroom on the same artifact.

6.5 C6: Daft Comparison

Level	Operation	Daft	`webfold .seed` 8-thread	Ratio
L0	parse and count all 254K WARC records	46,491 records/s	not measured	n/a

Level	Operation	Daft	`webfold .seed` 8-thread	Ratio
L1	parse and filter HTTP responses	13,494 pages/s	not measured	n/a
L2	parse, filter, extract HTML, render text	668 pages/s	26,675 pages/s	39.9x

Daft's native WARC reader is fast. The measured collapse occurs when full HTML body extraction and Markdown conversion are implemented as a Python `.apply()` UDF. The UDF path adds approximately 1.5 ms/page of interpreter overhead. This result supports a narrow conclusion: parser speed and full-pipeline extraction speed are distinct variables. The extraction boundary must remain in a compiled path if the target workload is full-corpus text conversion.

6.6 Scanner Ablation

`webfold` uses `hashtrinity::find_byte_from` and `find_pattern_from`, backed by explicit SIMD comparison and movemask operations, for the tag-search hot path.

Byte-scanning method	Pages/s on raw WARC, 8 threads	Notes
naive `iter().position()`	13,051	LLVM auto-vectorized baseline
hashtrinity explicit SIMD	13,978	position-table outer loop and SIMD `<` jumps
external `memchr` crate	14,238	tested, then reverted to avoid external dependency

The in-tree SIMD path improves the scanner while preserving dependency constraints. The external `memchr` result is included as a reference measurement; the selected implementation uses the in-tree scanner. The difference between 13,978 pages/s and 14,238 pages/s is small relative to the 39.9x Daft full-pipeline gap and the S3 network ceiling; the main system win comes from keeping the entire extraction/render path native and from avoiding unnecessary buffer construction.

6.7 Parallel `.seed` Scaling

Configuration	Pages/s	Processing time	RSS	Scaling vs 1-thread
`.seed` single-thread stream	2,943	29 s	35 MB	1.0x
`.seed` 8-thread buffer approach	15,745	5.38 s	4.8 GB	5.3x
`.seed` 8-thread carry approach	26,675	3.17 s	4.8 GB	9.1x

The buffer approach constructs large decompressed vectors per worker. The carry approach processes each decompressed block immediately and carries only the WARC-record tail, reducing memory-bandwidth pressure. The reported super-linear scaling relative to the single-thread stream is explained by the different data path: the parallel path operates over compressed seed blocks already resident in memory, while the single-thread stream reads through the file and buffered I/O stack block by block.

6.8 C8: Swarm-Collapse Validity

Swarm collapse is valid only for clustered update dynamics that converge. For an affine cluster update

$$S_{t+1} = AS_t + B,$$

the finite-dimensional contraction condition used here is $\rho(A) < 1$. When that condition holds, the limiting state is

$$S^* = (I - A)^{-1}B.$$

The reported local swarm-collapse note reduces 5,000 agents and 40 rounds to 8 clusters and one fixed-point pass, with approximately 10^{-7} residual in the contractive case. The step-count reduction relative to explicit iteration is approximately $5000 \times 40/8 = 25{,}000$. The result is conditional: if $\rho(A) \geq 1$, if the update is not stable under the observed state representation, or if the residual check fails, the cluster is not collapsed and must run by explicit simulation or model calls.

7. Integration: From Corpus to Research Loop

The combined research loop is:

```
inquiry
  -> typed acquisition schema
  -> live URL / CDP browser / WARC / seed / S3 object / PDF / ODT / CSV / XLSX
  -> webfold + document/table adapters
  -> DOM observations, HTTP bodies, Markdown blocks, chunks, tables, graph nodes
  -> syndesi session state
  -> model proposal
  -> threat scan and permission gate
  -> tool execution or MCP call
  -> result appended as a classified record
  -> optional swarm collapse for multi-agent fanout
```

The division is operationally important. `webfold` performs acquisition over live web, browser, archive, and document surfaces. The memory substrate stores the resulting observations as records, folds, branches, and graph edges. `syndesi` controls stateful action. Swarm collapse controls fanout when many agents share a contractive update. Results are interpreted at their own layer, with separate variables and refusal conditions.

A typical web-backed research run has four phases:

1. Define the typed acquisition schema from the inquiry.
2. Choose the acquisition path: live crawl/CDP, local WARC/seed, direct S3 stream, PDF/document conversion, or table extraction.
3. Normalize the output into Markdown blocks, typed rows, chunks, and graph records.
4. Run the guarded agent loop over selected chunks, records, and local tools.
5. If the plan fans out over repeated agent variants, cluster equivalent dynamics and collapse only convergent clusters.

This ordering keeps the expensive and side-effecting parts separate. Corpus extraction can be repeated and hashed. Live collection can preserve its schema, request log, and browser observations. Tool execution remains gated. Fanout is reduced only after the update dynamics are known.

8. Failure Conditions

Component	Failure condition	Required behavior
`syndesi` threat scan	span detected by a law	normalize before model and persistent records
`syndesi` permission gate	tool is denied or requires approval	block or pause loop
`syndesi` guardrail	pre-tool policy denies action	return tool error, no execution
`webfold` static extraction	page requires browser-equivalent JS execution	route to browser/CDP path
`webfold` browser/CDP path	requested browser feature is outside the implemented CDP subset	return an explicit unsupported-method error or route to a heavier browser backend
document/table adapters	unsupported file structure or malformed table	report adapter failure and preserve the raw source reference
`.seed` carry path	WARC record spans block boundary unrecovered	report page loss and use exact-coverage path when required
S3 ingestion	transport below extraction throughput	report network-bound ceiling
Daft comparison	UDF implementation differs	separate parser result from full extraction result
swarm collapse	$\rho(A) \geq 1$ or residual check fails	deopt to explicit simulation/model calls

These failure modes are part of the method. A system that reports page loss, network saturation, permission denial, or non-contractive dynamics is producing usable scientific evidence. A system that hides those conditions collapses distinct variables into an uninterpretable score.

9. Reproducibility Notes

The measurements in this manuscript should be reproduced with the following artifacts captured in a run directory:

Artifact	Required fields
machine profile	CPU, RAM, disk, GPU, kernel, virtualization, network RTT
corpus manifest	WARC size, record counts, response count, seed size, seed block count, object URI
command log	exact command, environment, thread count, stdout/stderr
raw counters	pages, bytes, wall time, CPU time, RSS, per-thread page counts
competitor versions	rclone, Daft, pyarrow, Python, any UDF code
output hashes	seed hash, WARC hash where feasible, Markdown-output hash or sampled checksums

The current benchmark used warm OS page cache for local reads. Cold-cache first-read performance should be measured separately. CPU frequency was not controlled from inside the guest. Thread placement was left to the OS scheduler. These factors should be reported with future runs rather than normalized away.

10. Discussion

The measured system has four distinct scaling laws.

First, acquisition scales by surface. Live web and browser/CDP acquisition scale with network latency, page behavior, DOM work, browser observation cost, and anti-automation constraints. Document acquisition scales with parser and layout complexity. WARC acquisition scales with bytes, record alignment, and memory traffic. The WARC benchmark in this paper measures the archive surface; it does not bound the browser or document surfaces.

Second, the best local WARC path is fast because it combines compressed seed input, block-parallel decompression, immediate extraction, and carry-based WARC alignment. The single-thread stream remains useful because it gives O(1) memory at 2,943 pages/s. Direct S3 streaming is the operational mode for object-store attached corpora; local staging is the operational mode for CPU-bound extraction benchmarks.

Third, research-agent action scales with tool-loop iterations and policy checks. In `syndesi`, a parsed action is a candidate transition. It must pass identity, authorization, guardrail, and permission checks before mutation.

Fourth, multi-agent research scales with the number of independent dynamic states. Swarm collapse can reduce repeated rounds only when clustered dynamics are contractive. The fixed-point solve applies to convergent iteration.

The most important systems consequence is that each layer can be optimized without changing the scientific claim of the others. Faster S3 transport would improve remote archive processing but would not change the policy semantics of `syndesi`. A stronger guardrail would change the set of legal tool transitions but would not change the WARC extraction rate. A new ODT or XLSX adapter would extend acquisition coverage while entering the same typed-record interface. Better cluster discovery would increase the number of agents eligible for collapse but would not weaken the contraction requirement.

11. Conclusion

The combined architecture gives a concrete research-agent stack: web and document acquisition through `webfold` and adjacent fold-family adapters, guarded tool execution through `syndesi`, typed memory through append/fold/graph records, and conditional multi-agent reduction through swarm collapse. The strongest measured result in this manuscript is the WARC benchmark: 26,675 pages/s on the `.seed` 8-thread carry path, 14,238 pages/s on raw WARC, and a 39.9x advantage over a Daft full-pipeline UDF path on the same corpus. The strongest source-level system property is that `syndesi` routes native and fallback tool calls through one execution and policy path. The swarm result is conditional: 5,000 agents and 40 rounds collapse to 8 fixed-point clusters only when the update is contractive and the residual check passes.

References

1. OpenAI, Codex documentation. https://learn.chatgpt.com/docs/cloud.
2. Anthropic, Claude Code overview. https://code.claude.com/docs/en/overview.
3. OpenCode documentation. https://opencode.ai/docs/.
4. OpenClaw repository. https://github.com/openclaw/openclaw.